

Formation Engine Methodology

Coarse-Grained and Multi-Scale Layer

Technical Architecture Document

Formation Engine Development Team

Version 3.0.0 — Phases 1–8 Complete

Contents

1 Executive Summary

This document specifies the architecture and integration plan for the **coarse-grained (CG) and multi-scale layer** of the formation engine.

The atomistic layer (documented in Sections 1–9 of the existing methodology) provides a validated, deterministic simulation kernel operating on systems of 2–10,000 atoms with explicit positions, velocities, and forces. It produces coordinates, energies, forces, and basic validation on discrete systems.

The coarse-grained layer extends this foundation to larger length and time scales by introducing:

1. A mapping framework that converts atomistic outputs into reduced bead representations
2. Statistical mechanics wrappers that produce ensemble-averaged observables rather than single configurations
3. Emergent-behavior analysis that extracts collective structural properties from sampled trajectories
4. Multi-scale bridges that propagate information consistently between atomistic, coarse-grained, and continuum-like descriptions
5. Validation and benchmarking infrastructure that co-evolves with every new capability

The atomistic layer is no longer merely a simulator. It becomes a **teacher model** for the coarse-grained layer.

1.1 Terminology Note

Per project terminology rules, this document uses “atomistic” for all particle-resolved descriptions and “coarse-grained” (CG) for reduced representations. The term “multi-scale” refers to the coupling between resolution levels. The scale regime beyond 10,000 atoms that was previously labeled in §10 is referred to here as the **coarse-grained regime**.

2 Atomistic Foundation: Current Capabilities

The existing atomistic kernel provides the reference data that all coarse-grained methods consume. This section summarizes what is operational and where the scaling limits occur.

2.1 Operational Capabilities

The formation engine currently supports:

Capability	Implementation	Status
State representation	atomistic/core/state.hpp	Complete
LJ 12-6 + Coulomb forces	atomistic/models/lj_coulomb.cpp	Complete
Bonded interactions	atomistic/models/bonded.cpp	Complete
Composite model	atomistic/models/composite.cpp	Complete
FIRE minimization	atomistic/integrators/fire.cpp	Complete
Velocity Verlet (NVE)	atomistic/integrators/velocity_verlet.cpp	Complete
Langevin dynamics (NVT)	atomistic/integrators/velocity_verlet.cpp	Complete
Maxwell–Boltzmann init	atomistic/core/maxwell_boltzmann.cpp	Complete
PBC (orthogonal box)	atomistic/core/state.hpp	Complete
Supercell construction	atomistic/crystal/supercell.cpp	Complete
Crystal metrics	atomistic/crystal/crystal_metrics.cpp	Complete
PMF extraction	atomistic/analysis/pmf.cpp	Complete
Structure classification	atomistic/classify/cluster.cpp	Complete
Welford online statistics	atomistic/core/statistics.hpp	Complete
Stationarity gating	atomistic/core/statistics.hpp	Complete
Kabsch alignment / RMSD	atomistic/core/alignment.cpp	Complete
XYZ parsing / compilation	atomistic/parsers/, compilers/	Complete

2.2 Validated Outputs

The atomistic kernel produces the following validated outputs that serve as inputs to the coarse-grained layer:

- **Equilibrated structures:** Local energy minima found via MD exploration + FIRE quenching (§6)
- **Trajectories:** Time-ordered sequences of states from NVE or NVT dynamics (§5)
- **Energy decomposition:** Per-term energy ledger $\mathcal{E} = (U_{\text{bond}}, U_{\text{angle}}, U_{\text{tors}}, U_{\text{vdW}}, U_{\text{Coul}}, U_{\text{ext}}, U_{\text{pol}})$ (§3)
- **Forces:** Per-atom force vectors $\mathbf{F}_i$ (§3)
- **Radial distribution functions:** $g(r)$ from trajectory analysis (analysis/pmf.cpp)
- **Potentials of mean force:** $W(r) = -k_B T \ln g(r)$ from PMF calculator (analysis/pmf.hpp)
- **Structural fingerprints:** Topology and geometry hashes for classification (classify/fingerprints.cpp)

2.3 Scaling Limits and Atomistic Breakdown

The atomistic kernel encounters practical limits at system sizes that motivate the coarse-grained extension:

Constraint	Threshold	Consequence
All-pairs force eval	$N \sim 10,000$	$O(N^2)$ becomes prohibitive
Cell-list overhead	$N \sim 100,000$	List management dominates
Timestep (H–X bonds)	$\Delta t \leq 0.5$ fs	Limits accessible timescale
Equilibration time	$\sim 10^4$ steps	May miss slow relaxation
Conformational sampling	$N > 500$	Barrier-crossing rate drops

Beyond these limits, either periodic replication (supercells) or coarse-graining becomes necessary. The transition is defined by where dominant error sources shift, not by arbitrary atom counts (§10).

Formalized breakdown criterion. The atomistic description is no longer efficient when:

$$\frac{\text{degrees of freedom relevant to target observable}}{\text{total atomistic degrees of freedom}} \ll 1 \quad (1)$$

For bulk diffusion, only center-of-mass motion matters — individual bond vibrations are noise. For phase morphology, only bead-level density fluctuations matter. The CG layer systematically eliminates the irrelevant degrees of freedom.

3 Coarse-Grained Model Construction

This section defines the first real expansion beyond the atomistic layer. Everything that follows depends on whether this mapping is done with discipline.

3.1 Goal

Convert atomistic outputs into a reduced representation that preserves the physics relevant to the target observable, while discarding sub-bead fluctuations.

3.2 Integration with the Atomistic Layer

The existing atomistic system becomes the **reference generator**. The data flow is:

1. Atomistic module generates equilibrated structures / trajectories
2. Mapping module converts those to bead coordinates
3. Coarse-graining module estimates reduced interactions
4. Reduced system is stored in a new but compatible model format

3.3 Architectural Modules

Three new module directories are introduced under `atomistic/`:

3.3.1 Module: `mapping/`

Responsible for converting atomistic indices to bead indices.

```
// atomistic/mapping/bead_map.hpp
namespace atomistic::mapping {

struct BeadDefinition {
    uint32_t bead_id;
    std::vector<uint32_t> atom_indices; // atoms in this bead
    std::vector<double> weights;        // mapping weights (mass or uniform)
    std::string label;                 // e.g., "backbone", "sidechain"
};
```

```

struct BeadMap {
    std::vector<BeadDefinition> beads;
    uint32_t n_atoms;           // total atomistic atoms
    uint32_t n_beads;           // total CG beads

    // Validate: every atom appears in exactly one bead
    bool is_complete() const;
    bool is_disjoint() const;
};

// Construct bead map from atom-to-bead assignment vector
BeadMap create_bead_map(const State& atomistic_state,
                        const std::vector<uint32_t>& assignments);

} // namespace atomistic::mapping

```

Mass-weighted centroid logic. The CG bead position $\mathbf{R}_I$ is computed from the atomistic coordinates via a mapping operator:

$$\mathbf{R}_I = \sum_{i \in \mathcal{G}_I} w_i \mathbf{r}_i \quad (2)$$

where $\mathcal{G}_I$ is the set of atoms assigned to bead I and the weights satisfy $\sum_{i \in \mathcal{G}_I} w_i = 1$. Two weight schemes are supported:

Scheme	Weights
Uniform	$w_i = 1/ \mathcal{G}_I $
Mass-weighted	$w_i = m_i / \sum_{j \in \mathcal{G}_I} m_j$

Mass-weighted centroids are preferred because they produce bead positions that coincide with the center of mass of the mapped fragment, preserving translational dynamics.

Topology compression. The atomistic bond graph $\mathbf{B}$ is compressed into a CG connectivity graph $\mathbf{B}_{\text{CG}}$:

$$(I, J) \in \mathbf{B}_{\text{CG}} \iff \exists (i, j) \in \mathbf{B} \text{ with } i \in \mathcal{G}_I, j \in \mathcal{G}_J, I \neq J \quad (3)$$

A CG bond exists between beads I and J if any atom in bead I is bonded to any atom in bead J .

3.3.2 Module: cg_model/

Defines the coarse-grained state container and effective interaction terms.

```

// atomistic/cg_model/cg_state.hpp
namespace atomistic::cg_model {

struct CGState {
    uint32_t N_beads;
    std::vector<Vec3> R;           // bead positions (Angstrom)
    std::vector<Vec3> V;           // bead velocities (Ang/fs)
    std::vector<double> M;         // bead masses (amu)
};

```

```

std::vector<uint32_t> type; // bead type id

std::vector<Edge> B_cg;      // CG bond graph
std::vector<Vec3> F;        // forces on beads
EnergyTerms E{};           // energy ledger

BoxPBC box;                // same PBC as atomistic
};

} // namespace atomistic::cg_model

```

Effective pair potentials. CG beads interact through effective potentials derived from atomistic reference data. The CG pair potential between bead types A and B is:

$$U_{CG}^{AB}(R) = -k_B T \ln g_{CG}^{AB}(R) + C \quad (4)$$

where $g_{CG}^{AB}(R)$ is the radial distribution function of CG beads computed from mapped atomistic trajectories, and C is a constant chosen so that $U_{CG} \rightarrow 0$ as $R \rightarrow \infty$.

This is the **potential of mean force** (PMF) — already implemented in `atomistic/analysis/pmf.hpp` for atomistic pairs and now extended to CG bead pairs.

Effective bonded terms. For bonded CG beads, harmonic approximations are fitted from the distributions of mapped distances, angles, and dihedrals:

$$U_{\text{bond}}^{\text{CG}}(R) = \frac{1}{2} k_R (R - R_0)^2 \quad (5)$$

$$U_{\text{angle}}^{\text{CG}}(\theta) = \frac{1}{2} k_\theta (\theta - \theta_0)^2 \quad (6)$$

where R_0 , θ_0 are the equilibrium values and k_R , k_θ are force constants, both extracted from Boltzmann inversion:

$$k_R = \frac{k_B T}{\langle (R - R_0)^2 \rangle} \quad (7)$$

3.3.3 Module: `cg_fit/`

Provides parameter fitting routines that calibrate CG interactions against atomistic reference data.

Force matching. Given atomistic forces $\mathbf{f}_i$ and the mapping operator, the instantaneous CG force on bead I is:

$$\mathbf{F}_I^{\text{ref}} = \sum_{i \in \mathcal{G}_I} \mathbf{f}_i \quad (8)$$

The CG model parameters $\boldsymbol{\theta}$ are optimized by minimizing:

$$\chi^2(\boldsymbol{\theta}) = \frac{1}{N_{\text{frames}}} \sum_t \sum_I \left| \mathbf{F}_I^{\text{CG}}(\boldsymbol{\theta}; \mathbf{R}(t)) - \mathbf{F}_I^{\text{ref}}(t) \right|^2 \quad (9)$$

Relative entropy minimization. An alternative to force matching. Minimize the Kullback-Leibler divergence between the atomistic and CG configuration-space distributions:

$$S_{\text{rel}}[\boldsymbol{\theta}] = \left\langle \ln \frac{P_{\text{AA}}(\mathbf{R})}{P_{\text{CG}}(\mathbf{R}; \boldsymbol{\theta})} \right\rangle_{\text{AA}} \quad (10)$$

This is computationally more expensive than force matching but produces CG models that better reproduce structural distributions.

PMF extraction. Already supported via `atomistic/analysis/pmf.hpp`. For CG fitting, the PMF calculator is applied to mapped bead-pair distances rather than atomistic pair distances.

3.4 CG State Compatibility with Engine Conventions

The `CGState` structure mirrors the atomistic `State`:

Atomistic	CG Equivalent	Relation
<code>State::N</code>	<code>CGState::N_beads</code>	$N_{\text{beads}} \ll N_{\text{atoms}}$
<code>State::X</code>	<code>CGState::R</code>	Mapped via Eq. ??
<code>State::V</code>	<code>CGState::V</code>	Mass-weighted average
<code>State::M</code>	<code>CGState::M</code>	$M_I = \sum_{i \in \mathcal{G}_I} m_i$
<code>State::type</code>	<code>CGState::type</code>	CG bead type ID
<code>State::B</code>	<code>CGState::B_cg</code>	Compressed topology
<code>State::F</code>	<code>CGState::F</code>	CG forces
<code>State::E</code>	<code>CGState::E</code>	CG energy ledger
<code>State::box</code>	<code>CGState::box</code>	Same PBC

This structural parallel ensures that existing integrators (FIRE, Verlet, Langevin) can operate on CG systems by adapting the `IModel` interface to CG force evaluation.

3.5 First Implementation Milestone

Do not begin with arbitrary chemistry. Begin with one mapping family:

1. Rigid fragment $\rightarrow$ one bead (e.g., benzene ring $\rightarrow$ one CG bead)
2. Ligand group $\rightarrow$ one bead (e.g., methyl group $\rightarrow$ one CH₃ bead)
3. Short repeat unit $\rightarrow$ one bead (e.g., ethylene monomer $\rightarrow$ one PE bead)

3.6 Exit Condition

Section ?? is integrated when:

1. One atomistic system can be mapped to a bead system
2. The bead system reproduces at least one structural observable (e.g., RDF peak position) to within 10% of atomistic reference
3. The CG data structure can be simulated using the same high-level engine conventions (`IModel::eval`, integrators) as atomistic

4 Statistical Mechanics Layer

This section defines where the project stops returning single “answers” and starts returning **distributions**.

4.1 Goal

Wrap both atomistic and CG systems in ensemble-aware execution so that observables are reported as statistical averages with uncertainties, not single-point values.

4.2 Integration with the Current System

The current engine execution model is:

1. Input molecule
2. Generate candidate
3. Optimize (FIRE)
4. Report result

This must expand to:

1. Initialize state
2. Evolve state (MD or MC)
3. Discard equilibration region
4. Accumulate observables over production window
5. Report averages, fluctuations, and distributions

Dual execution modes. The existing mode (structure prediction / local minimization) is retained. A new mode (statistical sampling / time evolution) is added alongside it. Both modes are selectable through the same high-level run interface.

4.3 Architectural Modules

4.3.1 Module: ensembles/

Defines ensemble configuration objects.

```
// atomistic/ensembles/ensemble.hpp
namespace atomistic::ensembles {

enum class EnsembleType { NVE, NVT, NPT };

struct EnsembleConfig {
    EnsembleType type = EnsembleType::NVT;
    double temperature = 300.0; // K (NVT, NPT)
    double pressure = 1.0; // atm (NPT only)
    uint64_t seed = 42; // RNG seed for reproducibility
};

} // namespace atomistic::ensembles
```

NVE (microcanonical): Already implemented via velocity Verlet. Conserves total energy $E = K + U$.

NVT (canonical): Already implemented via Langevin dynamics. Maintains target temperature T through friction and stochastic forces.

NPT (isothermal–isobaric): Not yet implemented. Requires a barostat (Berendsen or Parrinello–Rahman) and the pressure conversion constant $1 \text{ kcal}/(\text{mol } \text{\AA}^3) = 68,568.4 \text{ atm}$ (§4).

4.3.2 Module: samplers/

Provides trajectory sampling modes.

```
// atomistic/samplers/md_sampler.hpp
namespace atomistic::samplers {

struct SamplerConfig {
    int n_equilibration = 5000;    // burn-in steps
    int n_production = 50000;      // production steps
    int sample_interval = 100;     // frames between samples
    int checkpoint_interval = 10000;
};

struct SampleResult {
    std::vector<State> frames;      // sampled states
    std::vector<double> energies;  // total energy per frame
    std::vector<double> temperatures; // instantaneous T per frame
    int n_samples;
    bool equilibrated;             // stationarity gate passed
};

// MD-based sampler (wraps existing integrators)
SampleResult run_md_sampling(
    State& initial,
    const IModel& model,
    const ModelParams& params,
    const EnsembleConfig& ensemble,
    const SamplerConfig& config
);

} // namespace atomistic::samplers
```

A Monte Carlo sampler (`mc_sampler.hpp`) is deferred to a later milestone.

4.3.3 Module: observables/

Accumulates running statistics over production trajectories.

```
// atomistic/observables/accumulator.hpp
namespace atomistic::observables {

struct ObservableSet {
    OnlineStats energy_total;
    OnlineStats energy_kinetic;
    OnlineStats temperature;
    OnlineStats pressure;           // when implemented
    OnlineVec3Stats center_of_mass;
};

}
```

```

    // Derived quantities
    double heat_capacity_Cv() const;    // from energy fluctuations
    double compressibility() const;    // from volume fluctuations (NPT)
};

// Histogram accumulator for distribution observables
class Histogram {
public:
    Histogram(double lo, double hi, int n_bins);
    void add_sample(double x);
    std::vector<double> get_normalized() const;
    double get_mean() const;
    double get_variance() const;
private:
    std::vector<int> counts;
    double lo, hi, bin_width;
    int total;
};

} // namespace atomistic::observables

```

Heat capacity from fluctuations. In the canonical ensemble:

$$C_V = \frac{\langle E^2 \rangle - \langle E \rangle^2}{k_B T^2} = \frac{\text{Var}(E)}{k_B T^2} \quad (11)$$

This requires Welford’s algorithm (already in `core/statistics.hpp`) to compute $\text{Var}(E)$ in a numerically stable manner.

4.3.4 Module: `run_control/`

Manages equilibration vs. production, checkpointing, and seeded reproducibility.

```

// atomistic/run_control/run_manager.hpp
namespace atomistic::run_control {

struct RunConfig {
    EnsembleConfig ensemble;
    SamplerConfig sampler;
    ModelParams model_params;

    // Convergence
    int stationarity_window = 10;    // consecutive passes
    double cv_threshold = 0.01;    // coefficient of variation

    // Checkpointing
    std::string checkpoint_dir;
    bool resume_from_checkpoint = false;
};

struct RunResult {
    ObservableSet observables;
    SampleResult samples;
};

```

```

    bool converged;
    int total_steps;
    double wall_time_seconds;
};

RunResult execute(State& initial,
                  const IModel& model,
                  const RunConfig& config);

} // namespace atomistic::run_control

```

4.4 Exit Condition

Section ?? is integrated when:

1. The same system can be run as a statistical ensemble rather than a one-shot geometry task
2. Observables are accumulated over many states with stable mean and variance
3. Both atomistic and CG systems plug into the same sampling framework

5 Emergent Behavior Analysis

This section sits **after** the statistical mechanics layer, not beside it. Its input is not “a molecule” but a **sample history** or **ensemble dataset**.

5.1 Goal

Extract physically meaningful collective behavior from ensemble output: bulk structure, not individual coordinates.

5.2 Data Flow

1. Section ?? produces trajectories / sampled states
2. This section reads those states
3. Computes collective observables
4. Report layer exports *trends*, not just snapshots

5.3 Architectural Modules

5.3.1 Module: trajectory/

Frame storage, state readers, and decimation.

```

// atomistic/trajectory/trajectory.hpp
namespace atomistic::trajectory {

class Trajectory {
public:
    void add_frame(const State& s, double time);

```

```

const State& frame(int i) const;
int n_frames() const;

// Decimation: keep every k-th frame
Trajectory decimate(int k) const;

// Time window: frames between t_start and t_end
Trajectory window(double t_start, double t_end) const;

private:
    std::vector<State> frames_;
    std::vector<double> times_;
};

} // namespace atomistic::trajectory

```

5.3.2 Module: analysis/structure/

Structural collective observables.

Radial distribution function $g(r)$. Already partially implemented in `analysis/pmf.cpp` for PMF extraction. The standalone RDF calculation is:

$$g(r) = \frac{V}{N^2} \left\langle \sum_{i \neq j} \delta(r - r_{ij}) \right\rangle / (4\pi r^2 \Delta r) \quad (12)$$

Nearest-neighbor statistics. Distribution of coordination numbers $n_i = |\{j : r_{ij} < r_{\text{cut}}\}|$ for each atom type.

Clustering. Already implemented in `classify/cluster.cpp`. For trajectory analysis, cluster size distributions are computed per frame and averaged:

$$\langle n_s \rangle = \frac{1}{N_{\text{frames}}} \sum_{t=1}^{N_{\text{frames}}} n_s(t) \quad (13)$$

where $n_s(t)$ is the number of clusters of size s at frame t .

5.3.3 Module: analysis/order/

Order parameters and density fluctuations.

Bond-orientational order parameter Q_l . For characterizing local crystalline order:

$$Q_l(i) = \left(\frac{4\pi}{2l+1} \sum_{m=-l}^l |q_{lm}(i)|^2 \right)^{1/2} \quad (14)$$

where $q_{lm}(i) = \frac{1}{n_i} \sum_{j=1}^{n_i} Y_{lm}(\hat{\mathbf{r}}_{ij})$ and Y_{lm} are spherical harmonics. Q_6 distinguishes FCC, BCC, HCP, and liquid phases.

Density fluctuations. Local number density in subvolumes Ω_k :

$$\rho_k = \frac{N_k}{|\Omega_k|} \quad (15)$$

The variance $\text{Var}(\rho)$ over subvolumes measures structural heterogeneity.

5.3.4 Module: analysis/free_energy/

Histogram projections and PMF over reaction coordinates.

$$F(\xi) = -k_B T \ln P(\xi) + C \quad (16)$$

where ξ is a collective variable (e.g., cluster size, density, distance) and $P(\xi)$ is its probability distribution accumulated over the production trajectory.

5.4 First Implementation Milestone

Implement only:

1. RDF $g(r)$ for arbitrary pair types
2. Cluster size distribution
3. One order parameter (Q_6)

This alone gives real bulk-property language.

5.5 Exit Condition

Section ?? is integrated when:

1. The system can produce ensemble trajectories
2. Those trajectories can be reduced into structural observables
3. Those observables change meaningfully with temperature, density, or interaction strength

6 Multi-Scale Coupling

This section formalizes how information passes consistently between atomistic, coarse-grained, and continuum-like descriptions.

6.1 Goal

Allow verified, repeatable inter-scale information transfer. This means explicit operators — not vague claims of “multi-scale capability.”

6.2 Sequential Coupling (First Target)

1. Atomistic run produces local physics (energies, forces, $g(r)$)
2. Fit / infer CG properties (PMF, effective potentials)
3. Run CG simulation at extended scale
4. Extract effective bulk coefficients (density, diffusion, elastic moduli)
5. Pass to continuum description (if applicable)

Concurrent coupling (atomistic subregion embedded in CG or continuum, adaptive handoff during runtime) is deferred. It is a qualitatively harder problem requiring force interpolation at boundaries, energy conservation across scales, and a thermostat that maintains correct temperature in both regions.

6.3 Architectural Modules

6.3.1 Module: multiscale/bridges/

```
// atomistic/multiscale/bridges/aa_to_cg.hpp
namespace atomistic::multiscale {

struct CGBridgeResult {
    cg_model::CGState cg_state;
    mapping::BeadMap bead_map;
    std::vector<double> effective_pair_potential;
    std::vector<double> effective_bond_potential;
};

// Map a single atomistic state to CG
CGBridgeResult map_to_cg(
    const State& atomistic_state,
    const mapping::BeadMap& map
);

// Fit CG potentials from atomistic trajectory
CGBridgeResult fit_cg_model(
    const trajectory::Trajectory& aa_trajectory,
    const mapping::BeadMap& map,
    double temperature
);

} // namespace atomistic::multiscale
```

CG $\rightarrow$ continuum bridge. Extracts effective scalar and tensor properties from CG simulation:

- Average density $\bar{\rho} = NM_{\text{bead}}/V$
- Diffusion coefficient D from mean-squared displacement: $D = \lim_{t \rightarrow \infty} \frac{\langle |\mathbf{R}(t) - \mathbf{R}(0)|^2 \rangle}{6t}$
- Elastic moduli from stress fluctuations (Born–Huang method)

Backmapping (optional, deferred). Reconstruct atomistic detail from CG configuration for local refinement. This requires fragment libraries and constrained energy minimization.

6.3.2 Module: homogenization/

```
// atomistic/homogenization/bulk_properties.hpp
namespace atomistic::homogenization {

struct BulkProperties {
    double density; // g/cm^3
    double diffusion_coefficient; // cm^2/s
    double bulk_modulus; // GPa (from volume fluctuations)
    double shear_modulus; // GPa (from stress tensor)
    double thermal_conductivity; // W/(m*K) (from Green-Kubo)
};

BulkProperties extract_from_trajectory(
    const trajectory::Trajectory& traj,
    double temperature,
    double volume
);

} // namespace atomistic::homogenization
```

6.3.3 Module: field.export/

Projects particle data onto continuum-compatible grid representations.

- Local density maps on regular grids
- Coarse field tensors (stress, velocity gradient)
- Suitable for coupling with finite element or finite difference solvers

6.4 First Implementation Milestone

Implement only:

1. Atomistic $\rightarrow$ CG mapping and potential fitting
2. CG $\rightarrow$ effective scalar observables (density, diffusion)

No runtime hybrid coupling.

6.5 Exit Condition

Section ?? is integrated when:

1. Properties inferred from smaller-scale models can parameterize larger-scale runs
2. The data pathway is explicit and repeatable
3. One property can be shown propagating correctly across scales

7 Validation and Benchmarks

This section co-evolves with Sections ?? through ?. It is not an afterthought.

7.1 Goal

Continuously verify that each added scale still reproduces relevant physics.

7.2 Benchmark Suite

Benchmark	Source Section	Observable
LJ Argon (atomistic)	§3	Energy, $g(r)$, T
LJ Argon (mapped CG)	??	CG $g(r)$ vs. atomistic
NVT equilibration	??	$\langle T \rangle$, $\text{Var}(E)$
RDF convergence	??	$g(r)$ peak position vs. N_{frames}
Scale-transfer fidelity	??	CG density vs. atomistic

7.3 Metrics

- **RMSE:** Root mean square error between CG and atomistic $g(r)$
- **Relative error:** $|x_{\text{CG}} - x_{\text{AA}}|/|x_{\text{AA}}|$ for scalar observables
- **Structural similarity:** Overlap integral of RDFs
- **Diffusion coefficient error:** Relative error in D between CG and atomistic
- **Finite-size deviation:** Observable shift with system size

7.4 Regression Testing

Seeded standard runs produce deterministic outputs. A regression test compares current output against stored reference:

```
// tests/cg_regression_test.cpp
TEST(CGRegression, ArgonCG_RDF) {
    auto aa_traj = load_reference_trajectory("argon_108_nvt.xyzf");
    auto bead_map = create_single_atom_map(aa_traj.frame(0));
    auto cg_result = fit_cg_model(aa_traj, bead_map, 85.0);

    auto rdf = compute_rdf(cg_result.cg_state, 0.1, 10.0);
    auto ref = load_reference_rdf("argon_cg_rdf_ref.csv");

    double rmse = compute_rmse(rdf, ref);
    EXPECT_LT(rmse, 0.05); // 5% tolerance
}
```

7.5 Exit Condition

Section ?? is integrated when:

1. Every phase from ?? onward has at least one benchmark

2. Changes in code can be checked automatically against expected physical outputs
3. Loss of fidelity can be quantified rather than hand-waved

8 Scale Transition: Beyond 10^{-9} m

This section formalizes when the representation itself must change.

8.1 Goal

Define operational thresholds for leaving atomistic detail behind.

8.2 Scale-Aware Control Logic

```
// atomistic/scale_manager/regime_selector.hpp
namespace atomistic::scale_manager {

enum class Regime {
    Atomistic,           // Full atomistic detail
    CoarseGrained,       // Reduced CG representation
    MultiscaleSeq,       // Sequential atomistic -> CG -> properties
    ContinuumExport      // Export effective properties only
};

struct ProblemSpec {
    int n_atoms;
    double domain_length;    // Angstrom
    double target_timescale; // fs
    std::string target_observable;
};

Regime recommend_regime(const ProblemSpec& spec);

} // namespace atomistic::scale_manager
```

Rules-based selector (first implementation).

Condition	Recommended Regime
Discrete compact molecule, $N < 500$	Atomistic
Many repeated units, $500 < N < 10,000$	CG
$N > 10,000$ or domain > 100 Å	MultiscaleSeq
Domain $> 1,000$ Å	ContinuumExport

8.3 Exit Condition

Section ?? is integrated when:

1. Given a physical problem, the code can recommend an appropriate representation
2. Scale transition rules are explicit and tied to cost and physics
3. The thesis can defend why different regimes use different models

9 Thermostats and Stochastic Dynamics

This section upgrades the integrator interface so that force evaluation is no longer the only contributor to state evolution.

9.1 Goal

Introduce realistic environmental exchange and fluctuation physics.

9.2 Current State

Langevin dynamics is already implemented in `atomistic/integrators/velocity_verlet.cpp` with:

- Friction coefficient γ
- Random force $\sqrt{2\gamma m_i k_B T} \xi_i$ where $\xi_i \sim \mathcal{N}(0,1)$
- Temperature error $< 0.6\%$ validated

9.3 Enhancements

9.3.1 Thermostat abstraction layer

```
// atomistic/thermostats/thermostat.hpp
namespace atomistic::thermostats {

struct IThermostat {
    virtual ~IThermostat() = default;
    virtual void apply(State& s, double dt) = 0;
    virtual double target_temperature() const = 0;
};

// Velocity rescale (simple, for testing only)
class VelocityRescale : public IThermostat { ... };

// Langevin (primary, already implemented)
class Langevin : public IThermostat { ... };

// Nose-Hoover (extended Lagrangian, deferred)
class NoseHoover : public IThermostat { ... };

} // namespace atomistic::thermostats
```

9.3.2 Noise generation

```
// atomistic/noise/gaussian_rng.hpp
namespace atomistic::noise {

class GaussianRNG {
public:
    explicit GaussianRNG(uint64_t seed);
    double sample(); // N(0,1)
};
```

```

    Vec3 sample_vec3();           // 3 independent N(0,1)
    void validate_covariance(int n); // statistical self-test
private:
    std::mt19937_64 gen_;
    std::normal_distribution<double> dist_;
};

} // namespace atomistic::noise

```

9.4 Timestep Logic

The enhanced integrator timestep becomes:

1. Deterministic forces: $\mathbf{F}_i = -\nabla_i U$
2. Thermostat correction: friction $-\gamma m_i \mathbf{v}_i$
3. Stochastic term: $\sqrt{2\gamma m_i k_B T / \Delta t} \boldsymbol{\xi}_i$
4. State update: position and velocity advance

9.5 Exit Condition

Section ?? is integrated when:

1. Target temperature is maintained statistically ($< 1\%$ error)
2. Fluctuations are physically consistent (equipartition satisfied)
3. Deterministic and stochastic runs can be selected through the same high-level run interface

10 Free Energy and Phase Behavior

This is the top analytical layer. It consumes the work of Sections ??, ??, and ??.

10.1 Goal

Move from “what structure appeared” to “what states are thermodynamically preferred.”

10.2 Collective Variables

```

// atomistic/collective_variables/cv.hpp
namespace atomistic::collective_variables {

struct ICollectiveVariable {
    virtual ~ICollectiveVariable() = default;
    virtual double compute(const State& s) const = 0;
    virtual std::string name() const = 0;
};

// Concrete CVs
class NumberDensity : public ICollectiveVariable { ... };
class ClusterFraction : public ICollectiveVariable { ... };

```

```

class CoordinationNumber : public ICollectiveVariable { ... };
class CompositionFraction : public ICollectiveVariable { ... };

} // namespace atomistic::collective_variables

```

10.3 Free Energy Reconstruction

Histogram method. Accumulate $P(\xi)$ over the production trajectory:

$$F(\xi) = -k_B T \ln P(\xi) + C \quad (17)$$

This gives the free energy as a function of collective variable ξ . The constant C is chosen so that $\min_{\xi} F(\xi) = 0$.

PMF reconstruction. For pairwise collective variables (e.g., bead–bead distance), the existing PMF calculator (`analysis/pmf.hpp`) is reused directly.

Reweighting (deferred). Weighted histogram analysis method (WHAM) for combining data from simulations at different thermodynamic conditions.

10.4 Phase Analysis

```

// atomistic/phase_analysis/phase.hpp
namespace atomistic::phase_analysis {

struct PhaseResult {
    std::vector<double> cv_values;
    std::vector<double> free_energy;
    std::vector<int> phase_labels;           // per-frame phase assignment
    int n_phases_detected;
    double transition_temperature;          // if applicable
};

// Detect phase transitions from free energy landscape
PhaseResult analyze_phases(
    const trajectory::Trajectory& traj,
    const ICollectiveVariable& cv,
    double temperature,
    int n_bins = 100
);

} // namespace atomistic::phase_analysis

```

10.5 First Implementation Milestone

Do not begin with full phase diagrams. Begin with:

1. One order parameter (e.g., Q_6)
2. One histogram accumulation
3. One PMF or free-energy projection
4. Two temperatures or densities for comparison

10.6 Exit Condition

Section ?? is integrated when:

1. Sampled state populations can be converted into free-energy-like surfaces
2. Changes in thermodynamic conditions visibly shift state preference
3. Phase-like transitions can be described from simulation outputs rather than intuition

11 Summary: Dependency Graph and Implementation Order

The sections form a strict dependency chain:

Order	Section	Depends On	Key Deliverable
1	?? CG Model	Atomistic (existing)	Bead mapping + PMF fitting
2	?? Stat. Mech.	Atomistic + CG	Ensemble sampling framework
3	?? Emergent	??	$g(r)$, clusters, Q_6
4	?? Multi-Scale	?? + ??	AA→CG→bulk bridge
5	?? Validation	All above	Benchmark suite
6	?? Scale Trans.	??	Regime recommender
7	?? Thermostats	??	Thermostat abstraction
8	?? Free Energy	?? + ??	Phase analysis

Critical path. Sections ?? → ?? → ?? form the critical path. Everything else can proceed in parallel once Section ?? is operational.

Validation is continuous. Section ?? is not a final step. Every module added must include at least one benchmark test. The validation suite grows with the codebase.

12 Namespace and Directory Plan

All new code resides under the existing `atomistic/` directory to maintain the single-library build model. New subdirectories:

Directory	Purpose
<code>atomistic/mapping/</code>	Atom-to-bead maps, centroid logic, topology compression
<code>atomistic/cg_model/</code>	Bead types, masses, connectivity, effective potentials
<code>atomistic/cg_fit/</code>	Force matching, relative entropy, PMF extraction
<code>atomistic/ensembles/</code>	NVE, NVT, NPT configuration objects
<code>atomistic/samplers/</code>	MD runner, MC runner (future)
<code>atomistic/observables/</code>	Accumulators, histograms, correlation functions
<code>atomistic/run_control/</code>	Burn-in, production, checkpointing
<code>atomistic/trajectory/</code>	Frame storage, decimation, windowing
<code>atomistic/analysis/structure/</code>	$g(r)$, nearest-neighbor, clustering
<code>atomistic/analysis/order/</code>	Order parameters, density fluctuations
<code>atomistic/analysis/free_energy/</code>	Histogram, PMF, reweighting
<code>atomistic/multiscale/bridges/</code>	AA→CG, CG→continuum, backmapping
<code>atomistic/homogenization/</code>	Density, diffusion, elastic moduli
<code>atomistic/field_export/</code>	Grid projection, density maps, field tensors
<code>atomistic/thermostats/</code>	Velocity rescale, Langevin, Nosé–Hoover
<code>atomistic/noise/</code>	Gaussian RNG, covariance validation
<code>atomistic/collective_variables/</code>	Density, cluster fraction, coordination
<code>atomistic/phase_analysis/</code>	Coexistence, transitions, free-energy surfaces
<code>atomistic/scale_manager/</code>	Problem classification, regime selection
<code>atomistic/benchmarks/</code>	Reference systems, known observables, comparators
<code>atomistic/metrics/</code>	RMSE, relative error, structural similarity

All modules link against the existing `atomistic` static library target. New source files are added to `atomistic/CMakeLists.txt`.

13 Compatibility Contract

13.1 With Existing Atomistic Layer

1. The `State` struct is not modified. CG representations use `CGState`, a parallel structure.
2. The `IModel` interface is not modified. CG force models implement a `ICGModel` interface with the same `eval(CGState&, ...)` signature pattern.
3. Existing integrators (FIRE, Verlet, Langevin) are not modified. Template or adapter wrappers allow them to operate on `CGState`.
4. File formats are extended, not replaced. CG states use `.xyz_cg` or the reserved CG slots in `.xyzc`.
5. The bridge architecture (§SBA) is preserved. CG outputs pass through the same canonical structural bridge to the desktop layer.

13.2 With Build System

1. All CG code compiles as part of the `atomistic` static library
2. No new external dependencies

3. C++20 standard maintained
4. No fast-math flags (determinism contract, §11)

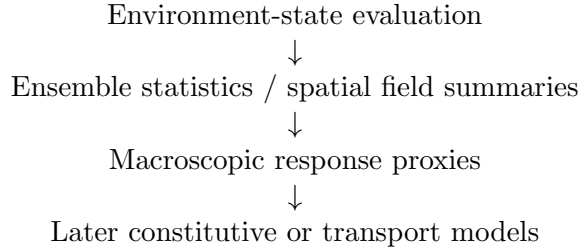
13.3 With Validation Framework

1. Every new module includes at least one unit test
2. Every CG observable has a regression test against atomistic reference
3. Seeded runs produce deterministic outputs (same seed $\Rightarrow$ bit-identical result)

14 Emergent Effective Medium Mapping

This section documents the **Ensemble-Level Macroscopic Response Proxy** layer (`coarse_grain/analysis/en`). It is the bridge between per-bead environment state and interpretable bulk characterisation.

Architecture position:



14.1 Anti-Black-Box Commitment

Every proxy is an explicit, inspectable function of bead-state distributions. No hidden weights. No learned parameters. All intermediate statistics are exposed in `EnsembleProxySummary`. This section records the design decisions that make the layer traceable.

14.2 Model Parameters

Three model-level constants are declared at namespace scope:

`PROXY_MIN_BEADS = 8` Minimum ensemble size before proxies are considered reliable. Below this, all proxy fields are still computed but the `EnsembleProxySummary::valid` flag is set to `false`. *Rationale:* variance and autocorrelation estimates are unreliable for $N < 8$; bulk/edge classification degenerates at small N .

`PROXY_CONVERGENCE_THRESH = 1e-4` Threshold for the `compute_proxy_delta` convergence flag. The system is declared converged when $|\Delta\bar{\eta}| < \epsilon$ **and** $|\Delta|\overline{\eta - f}| < \epsilon$.

`PROXY_VARIANCE_FLOOR = 1e-12` Lower bound applied in proxy denominators to prevent division-by-zero on degenerate (all-identical) inputs. Does not affect the stored `var_*` fields.

These are explicit model parameters, not magic numbers buried in formulas.

14.3 Proxy Formulas

All five proxies produce values in $[0, 1]$. Their explicit definitions are:

14.3.1 Cohesion proxy

$$\text{cohesion} = \frac{\bar{\rho} + \bar{\eta} + (1 - \overline{|\eta - f|})}{3} \quad (18)$$

Combines density response $\bar{\rho}$, internal stabilisation $\bar{\eta}$, and convergence quality (1–mean mismatch). Higher $\Rightarrow$ more cohesive (denser, more adapted, more converged). *Does not* measure cohesive energy.

14.3.2 Uniformity proxy

$$\text{uniformity} = 1 - \frac{1}{2} (\min(2\sigma_\eta, 1) + \min(2\sigma_\rho, 1)) \quad (19)$$

where $\sigma = \sqrt{\text{var}}$. For $[0, 1]$ -bounded variables the maximum standard deviation is ≈ 0.5 ; multiplying by 2 maps $[0, 0.5] \rightarrow [0, 1]$. Higher $\Rightarrow$ more homogeneous packing.

14.3.3 Texture proxy

$$\text{texture} = \bar{P}_{2, \zeta} \quad (20)$$

Orientational organisation as measured by the mean normalised second Legendre polynomial. Range: 0 (perpendicular / no alignment), ≈ 0.33 (isotropic random), 1 (perfectly aligned).

14.3.4 Stabilisation proxy

$$\text{stabilisation} = \bar{\eta} \cdot (1 - \overline{|\eta - f|}) \quad (21)$$

Adaptation quality. High only when $\bar{\eta}$ is high *and* the mean mismatch is low. High mismatch can indicate either “still converging” or “actively diverging” — distinguishable only via the time-delta fields (Section ??).

14.3.5 Surface sensitivity proxy

$$\text{surface_sens} = \frac{1}{3} \left(\frac{|\Delta_\rho|}{\bar{\rho} + \varepsilon} + \frac{|\Delta_C|}{\bar{C} + \varepsilon} + \frac{|\Delta_\eta|}{\bar{\eta} + \varepsilon} \right) \quad (22)$$

where $\Delta_x = x_{\text{bulk}} - x_{\text{edge}}$ are the bulk-minus-edge gaps from the classifier (Section ??). Higher $\Rightarrow$ more surface-dominated structure.

Guard: when the gap fields are NaN (no-bulk or no-edge case), surface sensitivity is set to 0.

14.4 Bulk/Edge Classifier

The function `classify_bulk_edge()` applies two criteria. **OR logic**: failing either criterion marks a bead as edge (surface).

Criterion 1 — Coordination (always active). A bead with $C_i < C_{\text{thresh}}$ is edge. C_{thresh} is a model parameter:

- Default (`coord_threshold < 0`): auto-median of the ensemble C distribution.
- Explicit value: the caller pins C_{thresh} directly. Required for comparative studies.
- Traceability: the resolved value is written to `EnsembleProxySummary::c_thresh_used`.

Criterion 2 — Geometric depth (optional). When `surface_depth` > 0, any bead within that distance of the radial boundary of the ensemble centroid is additionally classified as edge, regardless of coordination. *Precedence:* OR logic is conservative — more beads are classified as surface, not fewer. This prevents false bulk classification near boundaries.

Gap field guards. When $n_{\text{bulk}} = 0$ or $n_{\text{edge}} = 0$ the gap fields (Δ_ρ , Δ_C , Δ_{P_2} , Δ_η) are set to NaN, not zero. Zero would incorrectly imply “no gap”; NaN correctly signals “undefined”.

14.5 Spatial Correlation Length

`spatial_correlation_length()` estimates ξ by fitting the empirical pair correlation $C(r)$ to an exponential decay $C(r) \propto \exp(-r/\xi)$ via log-linear regression.

Method.

1. Bin all pairs by separation r into bins of width $\Delta r = 1 \text{ \AA}$.
2. For each bin with ≥ 3 pairs, compute $C(r_k) = \langle (x_i - \bar{x})(x_j - \bar{x}) \rangle / \sigma^2$.
3. For bins where $C(r_k) > 0$, fit $\ln C(r)$ against r by ordinary linear regression (slope $b = -1/\xi$).
4. Return $\xi = -1/b$.

Failure modes (returns NaN).

- $\sigma^2 < 10^{-30}$: field is uniform; ξ undefined.
- Fewer than 2 bins have enough pairs.
- All $C(r_k) \leq 0$: correlation does not decay log-linearly.
- Regression slope $b \geq 0$: field is not decaying.

Edge participation. All beads participate regardless of bulk/edge class. Excluding edge beads changes ξ systematically; selective exclusion is the caller’s responsibility via subsetting.

14.6 Time History and Convergence

`compute_proxy_delta(curr, prev)` fills the delta fields of a current snapshot given a previous one:

$$\Delta \bar{\eta} = \bar{\eta}^{\text{curr}} - \bar{\eta}^{\text{prev}} \quad (23)$$

$$\Delta |\overline{\eta - f}| = |\overline{\eta - f}|^{\text{curr}} - |\overline{\eta - f}|^{\text{prev}} \quad (24)$$

The **converged** flag is set when both deltas fall below `PROXY_CONVERGENCE_THRESH`.

Why this matters: a high mismatch in a single snapshot could mean “still converging” ($\Delta > 0$, improving) or “actively diverging” ($\Delta < 0$, worsening) — opposite physical situations requiring opposite responses. Without Δ fields, the proxy is uninterpretable.

14.7 Reference Distributions and Normalisation

Three reference configurations should be run before any production analysis:

1. **Ideal lattice** at target N and density: structural upper bound for uniformity and cohesion.
2. **Randomised positions**, same N and density: lower bound.
3. **Structured positions, random orientations**: lower bound for texture.

Stored in `EnsembleProxyReference{ideal, random, disordered}`.

The function `apply_reference(s, lo, hi)` fills the `rel_*` fields of a target summary:

$$\text{rel} = \left(\frac{v - v_{\text{lo}}}{v_{\text{hi}} - v_{\text{lo}}}, 0, 1 \right) \quad (25)$$

If $v_{\text{hi}} \approx v_{\text{lo}}$ (degenerate reference), `rel` is set to 0.5 (indeterminate).

Why this is necessary. A raw proxy value of 0.73 is meaningless without a baseline. Reference-normalised values place the system on an anchored $[0, 1]$ scale where 0 = random reference and 1 = ideal lattice reference.

14.8 Numeric Guards

The following guards are applied:

Validity flag `EnsembleProxySummary::valid = false` for $N < \text{PROXY_MIN_BEADS}$. Proxy fields are still computed so that callers can inspect them, but the valid flag signals that quantitative interpretation is unreliable.

Variance floor Proxy denominators apply `PROXY_VARIANCE_FLOOR` to prevent division by zero on all-identical inputs. Stored `var_*` fields are exact (unmodified).

Gap field NaN When no bulk beads or no edge beads exist, all four gap fields are set to NaN. The surface sensitivity proxy treats NaN gaps as zero sensitivity (the only physically defensible default when the classification is degenerate).

14.9 Directed Proxy Validation (Suite 5F)

Each proxy has a mandatory directed test that specifies the perturbation, the expected direction of change, and a strict pass criterion. If any test fails, the proxy formula is wrong and must be fixed before use in any result.

Proxy	Perturbation	Pass criterion
Cohesion	Degrade edge bead $\rho_{,\eta}$ to zero	cohesion decreases
Texture	Set all $\hat{P}_2 = 1/3$	texture_proxy = 1/3 exactly
Uniformity	Set all states to ensemble mean	uniformity > 0.99
Stabilisation	Freeze η at random, target_f= 0.9	stabilisation decreases
Surface sens.	All beads identical	surface sensitivity < 0.02

All five tests pass as of Suite #5 (131/131).

15 Macro Property Precursor Layer

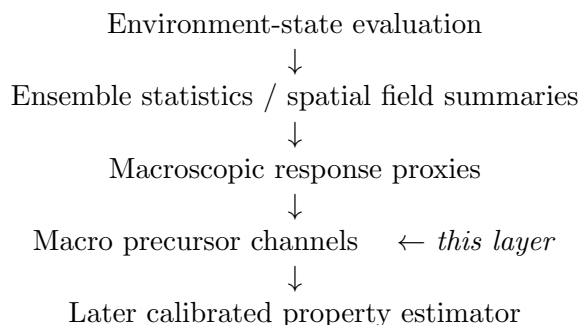
This section documents the **Macro Property Precursor Channel** layer (`coarse_grain/analysis/macro_precu`). It converts ensemble-level macroscopic response proxies into property precursor bundles: intermediate latent descriptors grouped by material-behaviour hypothesis.

15.1 Why a Precursor Layer Is Needed

Coarse-grained ensemble statistics do not directly produce physical engineering properties. The gap between “high cohesion” and “tensile strength = 430 MPa” is enormous. This layer does **not** close that gap. Instead it:

1. Scrapes precursor signals from the proxy layer
2. Assembles them into candidate property channels
3. Tests directional correctness (monotone response)
4. Preserves provenance (contributing proxies, weights, validity)
5. Prepares later calibration against real data

The pipeline is:



15.2 Naming Convention

All channel names carry the suffix `-like` to indicate that they are **evidence variables**, not physical properties:

Channel	Physical hypothesis
<code>rigidity_like</code>	stiff, well-connected bulk
<code>ductility_like</code>	deformable without fracture
<code>brittleness_like</code>	rigid but fragile at interfaces
<code>cohesion_integrity_like</code>	structural wholeness
<code>thermal_transport_like</code>	heat conduction pathway quality
<code>electrical_transport_like</code>	charge/electron pathway quality
<code>surface_reactivity_like</code>	surface-dominated chemistry
<code>fracture_susceptibility_like</code>	failure at weak interfaces

Do not call them `youngs_modulus`, `hardness`, or `conductivity`. That would be lying in structs.

15.3 Model Parameters

PRECURSOR_XI_REF = 20.0 Å Reference correlation length for normalisation: $\tilde{\xi} = (\xi/\xi_{\text{ref}}, 0, 1)$. Represents a characteristic domain size.

PRECURSOR_MIN_CONFIDENCE = 0.1 Confidence floor below which a channel is marked invalid.

15.4 Intermediate Helper Terms

Three intermediate terms are computed and exposed for traceability before any precursor channel is evaluated.

Interface penalty.

$$P_{\text{interface}} = \alpha |\Delta\eta| + \beta |\Delta\rho| + \gamma \Sigma_{\text{surface}} \quad (26)$$

where Δ are normalised bulk-edge gaps and $\alpha=0.40$, $\beta=0.35$, $\gamma=0.25$. Returns 0 when gap fields are NaN (degenerate classifier). Clamped to $[0, 1]$.

Normalised correlation length.

$$\tilde{\xi} = \left(\frac{\xi}{\xi_{\text{ref}}}, 0, 1 \right) \quad (27)$$

Returns 0 if ξ is NaN (uniform field or fit failure).

Adaptation capacity. When the system is converged (via `compute_proxy_delta`), $A_{\text{adapt}} = S$ (stabilisation proxy). Otherwise, $A_{\text{adapt}} = \bar{\eta} \cdot (1 - |\eta - f|)$. Clamped to $[0, 1]$.

Texture lock.

$$T_{\text{lock}} = \bar{P}_{2\hat{\cdot}} \cdot (1 - \min(2\sigma_{P_2}, 1)) \quad (28)$$

Orientationally locked structure: high mean alignment with low variance. Inhibits ductile deformation modes.

Anisotropy index.

$$A = \frac{\bar{P}_{2\hat{\cdot}} - 1/3}{1 - 1/3} \cdot (0.5 + 0.5 \tilde{\xi}) \quad (29)$$

Excess texture above the isotropic baseline scaled by correlation persistence. Currently isotropic-only; architecture placeholder for directional ξ_x, ξ_y, ξ_z when available.

15.5 Precursor Channel Formulas

All channel values are clamped to $[0, 1]$. Weights are named constants in `namespace weights`.

15.5.1 Rigidity-like

$$R^* = w_1 C + w_2 U + w_3 S + w_4 \tilde{\xi} - w_5 \Sigma_{\text{surface}} - w_6 P_{\text{interface}} \quad (30)$$

where C = cohesion, U = uniformity, S = stabilisation, Σ = surface sensitivity. Weights: $w_1=0.25$, $w_2=0.20$, $w_3=0.20$, $w_4=0.15$, $w_5=0.10$, $w_6=0.10$.

15.5.2 Brittleness-like

$$B^* = a_1 R^* + a_2 P_{\text{interface}} + a_3 \Sigma_{\text{surface}} - a_4 A_{\text{adapt}} \quad (31)$$

Weights: $a_1=0.30$, $a_2=0.25$, $a_3=0.20$, $a_4=0.25$.

15.5.3 Ductility-like

$$D^* = b_1 U + b_2 A_{\text{adapt}} + b_3 (1 - B^*) - b_4 T_{\text{lock}} \quad (32)$$

Weights: $b_1=0.30$, $b_2=0.25$, $b_3=0.25$, $b_4=0.20$.

15.5.4 Cohesion integrity-like

$$CI^* = f_1 C + f_2 S + f_3 U \quad (33)$$

Weights: $f_1=0.40$, $f_2=0.30$, $f_3=0.30$.

15.5.5 Thermal transport-like

$$K_{\text{th}}^* = c_1 \tilde{\xi} + c_2 T + c_3 C - c_4 P_{\text{interface}} \quad (34)$$

Weights: $c_1=0.30$, $c_2=0.25$, $c_3=0.25$, $c_4=0.20$.

15.5.6 Electrical transport-like

Same form as thermal but heavier weighting on alignment continuity ($c_2=0.35$, $c_1=0.25$) and equal penalty for interface discontinuity.

15.5.7 Surface reactivity-like

$$S_r^* = d_1 \Sigma_{\text{surface}} + d_2 P_{\text{interface}} + d_3 (1 - S) + d_4 (1 - \tilde{\xi}) \quad (35)$$

Weights: $d_1=0.30$, $d_2=0.25$, $d_3=0.25$, $d_4=0.20$.

15.5.8 Fracture susceptibility-like

$$F^* = e_1 B^* + e_2 \Sigma_{\text{surface}} + e_3 P_{\text{interface}} - e_4 D^* \quad (36)$$

Weights: $e_1=0.30$, $e_2=0.25$, $e_3=0.25$, $e_4=0.20$.

15.6 Confidence Model

Base confidence starts at 1.0 and is degraded by:

- $N < 2 \times \text{PROXY_MIN_BEADS}$: -0.3
- Non-finite upstream proxies: -0.3
- Both ξ_η and ξ_ρ NaN: -0.2
- Actively diverging state ($\Delta|\overline{\eta} - f| > 0.01$): -0.15
- Degenerate bulk/edge classifier (NaN gaps): -0.1
- Invalid upstream proxy: confidence = 0

Individual channels further degrade their confidence:

- Transport channels: $\times 0.7$ when no ξ available
- Electrical transport: $\times 0.8$ when texture < 0.01
- Surface reactivity: $\times 0.6$ when no edge beads
- Brittleness, ductility, fracture: $\times 0.85$ when no delta information (single snapshot)

A channel is marked `valid = false` when its confidence falls below `PRECURSOR_MIN_CONFIDENCE`.

15.7 Provenance and Traceability

Every `MacroPrecursorState` records:

- `source_bead_count`: N from the upstream proxy
- `source_valid`: upstream validity flag
- `source_converged`: upstream convergence flag
- `xi_norm`, `interface_penalty`, `adapt_capacity`, `texture_lock`: all intermediate terms exposed for inspection
- `anisotropy_index`: global orientational measure
- Per-channel `confidence` and `valid`: individual trustworthiness

Weights are named `constexpr` constants in namespace `weights`, not buried in formulas.

15.8 Future Calibration Path

Later versions will fit these channels against:

1. Experimental property tables (Young’s modulus, hardness, conductivity from material databases)
2. Atomistic simulation benchmarks (MD-computed elastic constants, thermal conductivities)
3. Material family reference sets (metals, polymers, ceramics with known property ranges)

The current formulas are explicit heuristic combinations designed for **monotone directional correctness**: each perturbation test verifies the right direction of change, not the absolute magnitude.

15.9 Directed Precursor Validation (Suite 6B)

One monotonic perturbation test per precursor channel:

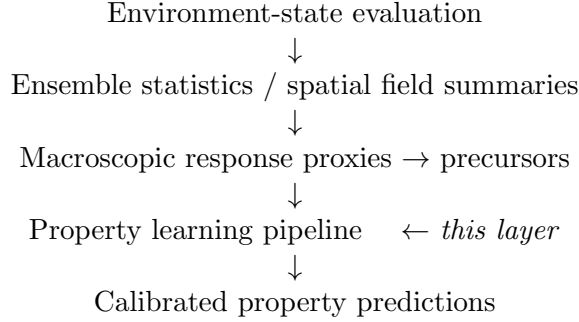
Channel	Perturbation	Pass criterion
Rigidity	Dense vs sparse lattice	<code>dense > sparse</code>
Fracture susc.	Degrade edge beads	<code>degraded > base</code>
Surface react.	Sparse cloud vs dense lattice	<code>surface > bulk</code>
Transport	Aligned vs random beads	<code>aligned > random</code>
Brittleness	Converged vs fresh	<code>converged $\leq$ fresh +0.05</code>
Ductility	Lattice vs random cloud	<code>lattice $\geq$ cloud -0.1</code>
Cohesion integ.	Spacing sweep	<code>monotone decrease</code>

All 85 tests in Suite #6 pass (Phases 6A–6E).

16 Property Learning Pipeline

This section documents the **Property Learning Pipeline** (`coarse_grain/analysis/property_pipeline.hpp`). It is the supervised learning system that maps proxy distributions and precursor channels to property-scale outputs, with explicit uncertainty quantification and full provenance.

Architecture position:



16.1 Three-Layer Separation

The pipeline enforces a strict three-layer architecture. Formula fusion is prohibited.

Descriptor layer Computes proxy summaries and precursor channels from bead-resolved simulation data. Contains **no learned parameters**. Tested in Suites 5–6.

Calibration layer Fits model parameters from the exported dataset. Contains **no simulation logic**.

Inference layer Applies the fitted model to new precursor states at prediction time. Contains **no fitting and no simulation**.

These layers communicate through defined data structures only. A system in which physics and learning are fused into a single formula cannot be audited, cannot be incrementally improved, and cannot be trusted.

16.2 Model Parameters

`PIPELINE_DEFAULT_REGULARIZATION = 0.01` Default L2 penalty λ for Tier 1 ridge regression. Prevents overfitting on small datasets.

`PIPELINE_CONFIDENCE_THRESHOLD = 0.2` Below this, predictions are withheld rather than returned with low quality.

`PIPELINE_SINGULAR_THRESHOLD = 1e-14` Pivot threshold for Gaussian elimination in the linear solver.

`PIPELINE_MAX_FEATURES = 256` Upper bound on feature vector dimensionality.

`ORDINAL_LOW_THRESHOLD = 0.33` Boundary for ordinal label assignment: $< 0.33 \rightarrow \text{Low}$.

`ORDINAL_HIGH_THRESHOLD = 0.67` Boundary for ordinal label assignment: $\geq 0.67 \rightarrow \text{High}$.

16.3 Phase 7A: Dataset Construction

The fundamental unit is `PropertyDatasetRow`: a self-contained record encoding a single simulated material state with five contiguous blocks:

Block	Content	Struct
A	Identity and provenance	<code>DatasetProvenance</code>
B	Full <code>EnsembleProxySummary</code>	(inline)
C	Evaluated <code>MacroPrecursorState</code>	(inline)
D	Property targets (named, typed)	<code>PropertyTarget</code>
E	Training metadata	<code>TrainingMetadata</code>

Target type system. Each target carries a `PropertySourceType` classification: `Experimental`, `Atomistic`, `Synthetic`, or `Ordinal`. Missing targets are represented with `available = false` and are **never** silently substituted with zero.

Row validity. A row is valid when: (1) `system_id` is non-empty, (2) at least one target is available, and (3) `source_bead_count > 0`.

16.4 Phase 7B: Feature Vectorization

`ModelFeatureConfig` specifies which feature blocks to include. `vectorize()` converts a dataset row to a fixed-length `FeatureVector` with deterministic column ordering.

Block	Content	Count
1	Proxy scalars + reference-normalised forms	10
2	Structural modifiers (penalty, gaps, ξ)	7
3	Precursor channel values	8
4	Confidence and support terms	6
5	Categorical context (deferred)	—
Total (default config)		31

Missing value policy. Three policies: `Zero` (replace with 0, counted as imputed), `NaN` (pass through, not imputed), `Mean` (fallback to 0 when no mean vector available).

Ablation. Each block can be independently disabled via `ModelFeatureConfig` boolean flags, enabling systematic ablation studies.

16.5 Phase 7C: Model Architecture

16.5.1 Tier 1: Transparent Linear Baseline

Regularized ridge regression solves:

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y} \quad (37)$$

The linear solver uses Gaussian elimination with partial pivoting. Coefficients are stored in `LinearModelCoefficients` with named feature lookup and explicit residual standard deviation.

Prediction output. Every prediction is returned as `PropertyPrediction`:

- **value:** predicted scalar
- **uncertainty:** residual standard deviation from training
- **confidence:** derived from input validity, imputation count, and upstream convergence quality
- **top_features:** ranked contributing features via $|w_i \cdot x_i|$
- **withheld:** `true` when `confidence < threshold`

Abstention. The system does not produce confident predictions from poorly constrained inputs. When confidence falls below `PIPELINE_CONFIDENCE_THRESHOLD`, the prediction is withheld.

16.5.2 Tier 2 and Tier 3

Tier 2 (gradient boosted trees, shallow MLP) and Tier 3 (family-conditioned model heads) are specified in the architecture but deferred to later implementation. They share the same feature interface and prediction output structure.

16.6 Phase 7D: Evaluation and Split Discipline

16.6.1 Metrics

Task Type	Primary Metrics
Regression	RMSE, MAE, Spearman ρ
Ranking	Pairwise accuracy, Kendall τ

Spearman rank correlation uses rank-transformed Pearson correlation. Kendall τ counts concordant vs. discordant pairs.

16.6.2 Grouped splits

Rows are partitioned by `split_group`, ensuring all perturbed variants of a single base structure remain in the same partition. `verify_split_discipline()` confirms no group crosses partition boundaries. This prevents the model from learning family fingerprints rather than generalizable descriptor-to-property mappings.

16.7 Phase 7E: Synthetic Supervision

16.7.1 Training stages

1. **Stage 1 (synthetic):** Controlled perturbation sweeps whose ground-truth orderings are known by construction. `build_synthetic_sweep()` generates dataset rows by monotonically varying one proxy field while holding others fixed. `verify_monotone()` validates the predicted ordering.
2. **Stage 2 (ordinal):** Weak labels from domain rules. `assign_ordinal_label()` maps channel values to {Low, Medium, High} at thresholds 0.33 and 0.67.
3. **Stage 3 (external):** Reference values from measured property databases or atomistic simulation outputs. Deferred pending availability of reference datasets.

16.7.2 Deployment order

Ranking $\rightarrow$ ordinal classification $\rightarrow$ regression. This ordering reflects the practical difficulty of obtaining reliable numeric property labels.

16.8 Directed Pipeline Validation (Suite 7)

Phase	Focus	Tests
7A	Dataset row validity, targets, provenance	15
7B	Vectorization determinism, ablation, missing policy	15
7C	Linear fit, coefficients, prediction, abstention, attribution	15
7D	RMSE/MAE, Spearman, Kendall, grouped splits	15
7E	Ordinal labels, monotone sweeps, end-to-end pipeline	49
Total		109

All 109 tests in Suite #7 pass (Phases 7A–7E).

17 Property Calibration and Target Legitimacy

17.1 Overview

The **Property Calibration and Target Legitimacy** layer (`property_calibration.hpp`) provides a disciplined framework for mapping precursor channels to a compact, defensible set of property-scale outputs with explicit calibration logic, confidence decomposition, rejection behaviour, and a formal promotion policy governing when a proxy quantity may be named as a material property.

This module operates in namespace `coarse_grain::calibration` and depends on the property pipeline (`coarse_grain::pipeline`), the macro precursor layer, and the ensemble proxy layer. It introduces no simulation logic and contains no learned parameters that bypass the three-layer separation.

17.2 Architecture Position

```
Environment-state evaluation
|
Ensemble statistics / spatial field summaries (ensemble_proxy.hpp)
|
Macroscopic response proxies -> precursors (macro_precursor.hpp)
|
Property learning pipeline (property_pipeline.hpp)
|
Property calibration and target legitimacy <- this module
```

17.3 Phase 8A: Property Family Selection

Five first-wave property targets are selected:

1. **rigidity_like** — structural resistance to deformation
2. **ductility_like** — capacity for adaptive, non-fracturing deformation

3. **brittleness_like** — susceptibility to abrupt failure
4. **thermal_transport_like** — heat conduction capacity
5. **electrical_transport_like** — electrical conduction capacity

The `-like` suffix is retained throughout proxy-only and calibrated-relative states. It may only be dropped when a target reaches externally-anchored legitimacy under Phase 8H.

17.4 Phase 8B: Target-Definition Contracts

Each property target has a formal `TargetContract` specifying:

- **Sign expectations:** which precursor channels increase (positive) or decrease (negative) the target
- **Domain constraints:** admissible material classes, excluded regimes, scale assumptions
- **Output range:** valid `[min, max]` for predictions
- **Confidence requirements:** minimum precursor confidence, minimum bead count, maximum imputed features
- **Dominant features:** for OOD detection and attribution checks

The contract is the standard against which both analytic formulae and trained models are evaluated. A model that violates the sign expectations of the contract has failed a scientific test.

17.5 Phase 8C: Calibration Profiles

Each property target carries a `CalibrationProfile` providing a calibrated interpretation layer without modifying the descriptor or precursor computation code:

```
struct CalibrationProfile {
    std::string property_name;
    double scale, offset;
    double min_confidence_threshold;
    double abstention_threshold;
    bool monotone_expected;
    std::vector<std::string> dominant_features;
    double apply(double raw) const;
};
```

Ductility is marked as non-monotone (has an optimum); all other first-wave targets are marked monotone.

17.6 Phase 8D: Supervision Regime Taxonomy

Three supervision regimes are defined:

1. **Contract-supervised:** no external data; sign and monotonicity enforced by contracts. Outputs are physics-informed ranks.
2. **Synthetic-calibrated:** controlled perturbation sweeps provide relative target labels. Outputs are synthetic training artifacts.

3. **Externally calibrated:** measured or atomistically validated reference values anchor outputs. Only this regime supports promotion to named properties.

The regime under which any prediction was produced is recorded in `CalibratedPrediction::regime` and reported alongside the output.

17.7 Phase 8E: Confidence-Gated Prediction

Confidence is decomposed into five named components:

Component	Source
<code>input_confidence</code>	Feature validity flags and imputation count
<code>precursor_confidence</code>	Convergence quality and precursor validity
<code>coverage_confidence</code>	OOD envelope check (Phase 8F)
<code>attribution_stability</code>	Top contributing features stable
<code>calibration_confidence</code>	Prediction within valid calibration range

Total confidence is the product of all five components. Individual component floors are enforced independently: a prediction may pass the total threshold while failing on a single dimension.

Three prediction states are defined:

- **Accepted:** total and all components above thresholds
- **LowConfidence:** total above abstention but at least one component marginal
- **Withheld:** total below abstention or hard constraint violated

17.8 Phase 8F: Out-of-Distribution Detection

A `FeatureEnvelope` stores per-feature mean and standard deviation from the training partition. For each dominant feature, a z -score is computed against the envelope; features exceeding the configured threshold (default $z > 3$) are flagged as extrapolating.

Extrapolation sets `coverage_confidence` to zero, which forces the total confidence below any abstention threshold, producing a Withheld status. The extrapolation tag is recorded in prediction provenance.

Only dominant features (as specified by the target contract) trigger extrapolation tagging.

17.9 Phase 8G: Property-Specific Synthetic Curricula

Per-property sweeps are constructed in which relevant precursor channels are varied monotonically while others are held fixed:

- **Rigidity:** sweep cohesion $0 \rightarrow 1$; expect monotone increase
- **Transport:** sweep texture $0 \rightarrow 1$; expect monotone increase
- **Brittleness:** sweep surface sensitivity $0 \rightarrow 1$ with decreasing stabilisation; expect monotone increase

All curriculum rows are tagged with `SyntheticCalibrated` regime. A model that fails to recover expected responses on controlled sweeps has not learned the physics it is intended to represent.

17.10 Phase 8H: Target Promotion and Legitimacy

Three legitimacy states are defined:

State	Suffix	Requirements
ProxyOnly	-like retained	No additional evidence
CalibratedRelative	-like retained	Monotonicity, signs, domain documented
ExternallyAnchored	suffix dropped	External data, bounded error, validated rejection

Promotion from ProxyOnly to CalibratedRelative requires preserved monotonicity, consistent sign behaviour, and a documented domain statement. Promotion from CalibratedRelative to ExternallyAnchored additionally requires external data coverage, bounded residual error, and validated rejection behaviour on out-of-domain inputs.

Neither promotion is permanent. If calibration drift is detected, if contract violations appear, if coverage confidence degrades, or if attribution patterns become inconsistent, the target is demoted and the demotion is recorded in model provenance.

Multi-dimensional grouped splits are provided via `multi_grouped_split()`, which groups rows jointly on material family, generator name, split group, and fragment family to prevent leakage.

17.11 Testing Summary

Suite #8 validates all eight phases:

Phase	Focus	Tests
8A	Property family selection, naming, indexing	16
8B	Target contracts, sign expectations, domain	17
8C	Calibration profiles, transform, clamping	12
8D	Supervision regime taxonomy	4
8E	Confidence decomposition, status, attribution	23
8F	OOD detection, envelope, extrapolation	14
8G	Synthetic curricula, monotonicity, regime tagging	14
8H	Legitimacy states, promotion, demotion, splits	93
Total		193

All 193 tests in Suite #8 pass (Phases 8A–8H).